

Correction détaillée – Évaluation : Suites géométriques

Terminale STI2D – Durée : 20 min

Calculatrice interdite : on garde les expressions exactes.

Exercice 1 – Formules (6 points)

On complète les formules pour une suite géométrique (u_n) de premier terme u_0 et de raison q .

1. $u_{n+1} = u_n \times q$.
2. $u_n = u_0 \times q^n$.
3. $S_N = u_0 + u_1 + \cdots + u_{N-1}$.
4. $S_N = u_0 \times \frac{1 - q^N}{1 - q} \quad (q \neq 1)$.
5. Si $q = 1$, alors $S_N = N \times u_0$.
6. Pour $N = 50$, on a $S_{50} = u_0 \times \frac{1 - q^{50}}{1 - q}$.

Exercice 2 – Calculs (8 points)

On considère $u_0 = 500$ et $q = 0,95$.

1. Calcul de u_{10} :

$$u_{10} = u_0 \times q^{10} = 500 \times 0,95^{10}.$$

On laisse cette expression exacte, la calculatrice étant interdite.

2. Calcul de S_{50} :

$$S_{50} = u_0 + u_1 + \cdots + u_{49} = u_0 \times \frac{1 - q^{50}}{1 - q}.$$

On remplace :

$$S_{50} = 500 \times \frac{1 - 0,95^{50}}{1 - 0,95} = 500 \times \frac{1 - 0,95^{50}}{0,05} = 10000(1 - 0,95^{50}).$$

On laisse également cette expression exacte.

Exercice 3 – Python (6 points)

On reprend $u_0 = 500$ et $q = 0,95$.

1. Programme A – à comprendre

```
def prog_01(u0, q, n):  
    u = u0  
    k = 0  
    while k < n:  
        u = u * q  
        k = k + 1  
    return u
```

- Le programme effectue n multiplications successives par q à partir de u_0 . Il renvoie donc $u_0 \times q^n$.
Pour $n = 10$, `prog_01(500, 0.95, 10)` renvoie $500 \times 0,95^{10}$. C'est exactement le calcul de u_{10} demandé à l'Exercice 2, question 1.
- Ce programme traduit la formule $u_n = u_0 \times q^n$ (Exercice 1, question 2).

2. Programme B – à compléter

```
def prog_02(u0, q, N):
    S = 0
    u = u0
    for k in range(N):
        S = S + u
        u = u * ----
    return S
```

- Le trou doit être complété par q . En effet, après avoir ajouté le terme courant u à la somme S , on passe au terme suivant en multipliant u par la raison q .
- `prog_02(500, 0.95, 50)` additionne les 50 premiers termes $u_0, u_1, \dots, u_{49}$. Il renvoie donc S_{50} . D'après l'Exercice 2,

$$S_{50} = 10000(1 - 0,95^{50}).$$

- Pour effectuer le même calcul sans boucle, on utilise la formule de la somme :

```
def prog_02_sans_boucle(u0, q, N):
    if q == 1:
        return N * u0
    else:
        return u0 * (1 - q**N) / (1 - q)
```

Si l'on sait que $q \neq 1$, on peut écrire plus simplement :

```
def prog_02_bis(u0, q, N):
    return u0 * (1 - q**N) / (1 - q)
```

L'appel `prog_02_sans_boucle(500, 0.95, 50)` renvoie alors

$$10000(1 - 0,95^{50}).$$

3. Programme C – à compléter avec `if ... elif`

```
def fonc_01(u_p, q, p):
    u_suivant = u_p * q
    if q > 0:
        if u_suivant > u_p:
            return "-----"
        elif u_suivant < u_p:
            return "-----"
        else:
            return "-----"
    else:
        return "___ monotone"
```

- Les mots manquants sont :
 - si $u_{suivant} > u_p$: **croissante** ;
 - si $u_{suivant} < u_p$: **décroissante** ;

- sinon : **constante** ;
 - dans le **else** : **non** (soit "non monotone").
- b. Ici, $q = 0,95 \times 2,25 = 2,1375$. Donc $q > 0$. On a $u_p = 500$, donc

$$u_{\text{suivant}} = u_p \times q = 500 \times 2,1375 = 1068,75.$$

Comme $u_{\text{suivant}} > u_p$, la fonction renvoie "croissante".

4. Programme D – à comprendre et à relier aux formules

```
def prog_03(u0, q):
    if q == 1:
        S = 50 * u0
    elif q > 0:
        S = 0
        u = u0
        for k in range(50):
            S = S + u
            u = u * q
    else:
        S = 0
        u = u0
        k = 0
        while k < 50:
            S = S + u
            u = u * q
            k = k + 1
    return S
```

- a. Le cas $q = 1$ est traité à part car la formule $S_N = u_0 \frac{1 - q^N}{1 - q}$ n'est pas définie pour $q = 1$ (division par zéro). Or, si $q = 1$, la suite est constante : $u_n = u_0$ pour tout n . Donc $S_{50} = 50 \times u_0$. Cela correspond à l'Exercice 1, question 5.
- b. La partie **elif q > 0** calcule S_{50} à l'aide d'une boucle **for** : on initialise S à 0, puis on ajoute u à S et on multiplie u par q , 50 fois. La partie **else** calcule exactement la même somme S_{50} , mais à l'aide d'une boucle **while**. Les deux méthodes donnent le même résultat.
- c. `prog_03(500, 0.95)` calcule

$$S_{50} = u_0 + u_1 + \cdots + u_{49}$$

pour $u_0 = 500$ et $q = 0,95$. Il renvoie donc

$$S_{50} = 10000(1 - 0,95^{50}).$$

Cela correspond à l'Exercice 2, question 2.

Méthode : dérouler un programme Python sans calculatrice

Lorsque la calculatrice est interdite, on ne cherche pas à calculer les puissances de 0,95. On garde les expressions exactes : $0,95^2$, $0,95^{10}$, $0,95^{50}$, etc. Le tableau de déroulement permet de suivre l'évolution des variables et de vérifier les conditions logiques (Vrai/Faux) à chaque étape.

Programme A : prog_01(500, 0.95, 10)

On rappelle le programme :

```
def prog_01(u0, q, n):  
    u = u0  
    k = 0  
    while k < n:  
        u = u * q  
        k = k + 1  
    return u
```

Ici : $u_0 = 500$, $q = 0,95$, $n = 10$.

Étape k	Condition $k < 10$	u avant	u après
0	Vrai	500	$500q$
1	Vrai	$500q$	$500q^2$
2	Vrai	$500q^2$	$500q^3$
3	Vrai	$500q^3$	$500q^4$
4	Vrai	$500q^4$	$500q^5$
5	Vrai	$500q^5$	$500q^6$
6	Vrai	$500q^6$	$500q^7$
7	Vrai	$500q^7$	$500q^8$
8	Vrai	$500q^8$	$500q^9$
9	Vrai	$500q^9$	$500q^{10}$
10	Faux	$500q^{10}$	arrêt

Le programme renvoie donc $u = 500q^{10}$. Avec $q = 0,95$, cela donne :

$$u_{10} = 500 \times 0,95^{10}.$$

C'est exactement le calcul demandé à l'Exercice 2, question 1.

Programme B : prog_02(500, 0.95, 50)

On complète le programme :

```
def prog_02(u0, q, N):  
    S = 0  
    u = u0  
    for k in range(N):  
        S = S + u  
        u = u * q  
    return S
```

Ici : $u_0 = 500$, $q = 0,95$, $N = 50$. La boucle `for k in range(50)` fait prendre à k les valeurs $0, 1, 2, \dots, 49$.

k	u avant	S avant	S après	u après
0	500	0	500	$500q$
1	$500q$	500	$500 + 500q$	$500q^2$
2	$500q^2$	$500 + 500q$	$500 + 500q + 500q^2$	$500q^3$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
49	$500q^{49}$	$\sum_{i=0}^{48} 500q^i$	$\sum_{i=0}^{49} 500q^i$	$500q^{50}$

À la fin, le programme renvoie

$$S = \sum_{i=0}^{49} 500q^i = 500 \times \frac{1 - q^{50}}{1 - q}.$$

Avec $q = 0,95$, on obtient :

$$S_{50} = 500 \times \frac{1 - 0,95^{50}}{0,05} = 10000(1 - 0,95^{50}).$$

On laisse cette expression exacte si la calculatrice est interdite.

Programme C : `fonc_01(u_p, q, p)`

On complète le programme :

```
def fonc_01(u_p, q, p):
    u_suivant = u_p * q
    if q > 0:
        if u_suivant > u_p:
            return "croissante"
        elif u_suivant < u_p:
            return "décroissante"
        else:
            return "constante"
    else:
        return "non monotone"
```

Pour $u_p = 500$, $q = 0,95 \times 2,25$ et $p = 10$:

$$q = 0,95 \times 2,25 = 2,1375.$$

On peut calculer $u_{suivant}$ à la main :

$$u_{suivant} = 500 \times 2,1375 = 1068,75.$$

u_p	q	$u_{suivant}$	$q > 0$	$u_{suivant} > u_p$	Retour
500	2,1375	1068,75	Vrai	Vrai	"croissante"

La fonction renvoie donc "croissante".

Programme D : prog_03(500, 0.95)

On rappelle le programme :

```
def prog_03(u0, q):  
    if q == 1:  
        S = 50 * u0  
    elif q > 0:  
        S = 0  
        u = u0  
        for k in range(50):  
            S = S + u  
            u = u * q  
    else:  
        S = 0  
        u = u0  
        k = 0  
        while k < 50:  
            S = S + u  
            u = u * q  
            k = k + 1  
    return S
```

Ici $u_0 = 500$ et $q = 0,95$. On teste d'abord les conditions :

Condition	Valeur	Branche choisie
$q == 1$	Faux	elif
$q > 0$	Vrai	

Le programme exécute donc la partie **elif** $q > 0$. Cette partie est identique au programme B : elle calcule la somme des 50 premiers termes à l'aide d'une boucle **for**. Le déroulement est donc le même que celui du tableau du Programme B.

À la fin, `prog_03(500, 0.95)` renvoie :

$$S_{50} = 10000(1 - 0,95^{50}).$$

Cela correspond à l'Exercice 2, question 2.

Barème indicatif : Exercice 1 : 6 pts ; Exercice 2 : 8 pts ; Exercice 3 : 6 pts.